25-Company Validation Run — Result (batch 1276)

Verdict: CLEAN COMPLETION FAILED. 12 of 25 companies completed (all first-attempt, all within the 35-minute execution budget); 13 companies failed after exhausting all 3 attempts with an identical isolated-child spawn abort. No immutable snapshot was written (the batch is held in `pending_review`; results were not saved). Isolation **containment** held throughout; the isolated-child **execution path** failed under the 25-company / 8-slot load. Cost and wall-clock stayed far inside their boundaries. **No rerun performed. Paused after reporting, per instruction.**

This document reports isolation reliability **separately** from workload completion, and reports retries and eventual completions **separately** from first-attempt completions, as required.

---

1. Run identity and provenance

Item	Value
Batch id	**1276**
Launch (UTC)	2026-10-12T01:24:46Z
Terminal state reached (UTC)	2026-10-12T02:18:03Z (`pending_review`)
Wall-clock	**≈ 53.3 min** (boundary: 90 min — within)
Build SHA (app)	`b676c2ddf37512dad0e18c6e812a4dc7e58a12a2`
Build SHA (worker)	`b676c2ddf37512dad0e18c6e812a4dc7e58a12a2`
Framework	18 (36 measures) · Workspace 3
Roster	25 companies, SHA-256 `761f65e8de-b27223454deaed464675e4db28f6d65fae3d-b5fca8a95babd1a7e8` (frozen pre-launch; see `release-evidence/VALIDATION25_ROSTER.md`)
Worker topology	**2 live replicas × concurrency 4 = 8 execution slots** (unchanged); 8 jobs claimed instantly at launch, confirming the ceiling
Enqueue response	`{success:true, batchId:1276, enqueued:25, requested:25}`

Deployment fingerprint recorded on the batch confirms `liveAppSha == liveWorkerSha == sourceSha == b676c2d` with `tsCount:0` and `executableGeneralisa-`

`tionCount:0` — the run executed on the **unchanged approved build**. No code, replica, or settings change was made. `list_id` is NULL on the batch row (the resume-analysis endpoint does not persist it); roster provenance is recorded separately in the frozen roster document.

---

2. Clean-completion criteria — PASS/FAIL

Criterion (from approval)	Result	Status
25 **first-attempt** completions	12 first-attempt completions	**FAIL**
Each within 35-min execution budget	12/12 completed jobs ≤ 35 min (max 33.5)	PASS *(for the 12 that completed)*
36 results per company (900 total)	36/36 for each of the 12 completed (432 rows); 0 rows for the 13 failed	**FAIL** (432/900)
Correct batch/attempt provenance	All 12 completions `attempts=1`; all 13 failures `attempts=3`; all tagged `batch_id=1276`	PASS
Immutable snapshot (`companies_count=25`)	**No `analysis_results` row written**; batch held in `pending_review`, results not saved	**FAIL**

Clean completion is definitively failed. A clean completion requires all 25 to complete first-attempt within budget with a 25-company immutable snapshot. 13 hard failures and the absence of a snapshot each independently fail the criterion. This is a measured failure of the completion criterion — **not** an inconclusive result.

---

3. First-attempt completions (reported separately)

All 12 completions were **first attempt** and **within the 35-minute execution budget** (0 over budget). Each produced exactly 36 measure rows, 0 abstentions.

company_id	Company	attempts	exec (min)	score rows
9	CSX Corporation	1	15.4	36
25	OTSUKA CORPORATION	1	15.6	36
21	Power Assets Holdings Limited	1	15.6	36
10	MAGNA INTERNATIONAL INC.	1	17.2	36
11	THE SOUTHERN COMPANY	1	18.5	36
14	DARDEN RESTAURANTS, INC.	1	19.8	36
20	Dominion Energy, Inc.	1	20.0	36
15	BCE INC.	1	20.7	36
19	CELESTICA INC.	1	24.3	36
29	Idemitsu Kosan Co.,Ltd.	1	24.5	36
24	AXA SA	1	25.1	36
30	BARRATT REDROW	1	33.5	36

No completion resulted from a retry — the retry mechanism produced **zero** eventual completions (see §5).

---

4. Failures (reported separately)

13 companies failed, each after **3 attempts (retry-exhausted)**, all with the identical terminal error:

> `Isolated job child exited unexpectedly (code=null signal=SIGABRT stage=spawn)`

company_id	Company	attempts	terminal error
12	S&P GLOBAL INC.	3	SIGABRT @ stage=spawn
13	MARTIN MARIETTA MATERIALS, INC.	3	SIGABRT @ stage=spawn
16	STRYKER CORPORA-TION	3	SIGABRT @ stage=spawn
17	AMPHENOL CORPOR-ATION	3	SIGABRT @ stage=spawn
18	AFLAC INCORPOR-ATED	3	SIGABRT @ stage=spawn
22	BNP PARIBAS SA	3	SIGABRT @ stage=spawn
23	ABB Ltd	3	SIGABRT @ stage=spawn
26	ARISTOCRAT LEISURE LIMITED	3	SIGABRT @ stage=spawn
27	ASSOCIATED BRITISH FOODS PLC	3	SIGABRT @ stage=spawn
28	BT GROUP PLC	3	SIGABRT @ stage=spawn
2566	COTERRA ENERGY	3	SIGABRT @ stage=spawn
2567	J M SMUCKER	3	SIGABRT @ stage=spawn
2568	FACTSET RESEARCH SYS.	3	SIGABRT @ stage=spawn

5. Retry and eventual-completion accounting (reported separately)

Quantity	Value
Jobs that required a retry	13 (the failed set)
Retry attempts consumed	3 per failed job (retry cap reached on every one)
Eventual completions produced by retries	**0**
First-attempt completions	12
First-attempt failures that later recovered	0

Every retry of a SIGABRT-spawn failure reproduced the same spawn abort; no failed company was recovered by the retry policy. The 13 failures are therefore both first-attempt failures and retry-exhausted failures.

---

6. Isolation reliability (reported separately from completion)

The required distinction between **containment** (does a child failure stay contained, leaving the supervisor and the rest of the batch intact?) and the **execution path** (can the isolated child actually run the job?) is reported explicitly below. These are two different properties and they had two different outcomes.

6.1 Containment — RELIABLE (held throughout)

Evidence	Observation
`[WORKER-ISOLATION]` breach/quarantine/revoke/kill lines	**0** across the entire run (01:24–02:22)
Supervisor survival	Parent heartbeat continuous; 12 jobs completed cleanly **interleaved** with the 13 failures
Parent memory	`[DIAG][event-loop] rss_mb` flat at **150–207 MB** throughout — no climb, no leak
Child-death handling	Every child abort was caught → `failJob transitioned=true batchTerminal=false` → retried per existing policy
LLM permits	Peaked at `held=7` during saturation, returned cleanly to `held=0 waiting=0 max=8`; no leak, no waiter backlog
Lock renewal	`[DIAG][bull-lock] locksRenewed` firing ~every 75 s; **no lock-renewal failure**
Orphans / stale / redelivery / cleanup	None observed (the only "stale" log matches are normal corpus-hygiene de-duplication, not stop conditions)

The isolation boundary did its job: a crashing child never contaminated the supervisor or another job, capacity was never wrongly re-granted, and the batch was correctly driven to a terminal `pending_review` state rather than hanging.

6.2 Isolated-child execution path — FAILED under load

13 isolated children could not successfully start. The job-isolation classifier (`server/lib/job-isolation.ts`) initialises `lastStage = "spawn"` and only advances it when the child posts its first progress message; `code=null signal=SIGABRT` with `killReason=null` maps to *"child died with no kill requested"* → the retriable "exited unexpectedly" error. So `stage=spawn` means the forked child **self-aborted during Node/tsx bootstrap, before posting any progress**. The supervisor never asked it to stop.

This is **not** a supervisor deadline termination (correct deadline kills set `killReason` , which was null), **not** a recovered provider/browser error, and **not** a containment breach. It is a hard failure of the isolated-child execution path.

---

7. Root cause — mechanism confirmed; underlying resource strongly supported

Confirmed from code + logs (established): the forked child aborts (SIGABRT) at bootstrap before its first stage message. The timing is a sharp threshold:

- Container resource-exhaustion errors begin at **01:36:27** — `pthread_create: Resource temporarily unavailable` (EAGAIN), later joined by `posix_spawn … Resource temporarily unavailable (11)` and `chromium: Cannot fork`.
- SIGABRT-at-spawn child aborts begin at **01:45:28** and run as a ~2-minute burst to 01:47:14 — i.e. **after** the container was already thread-starved.
- Jobs whose children were forked **before** the threshold ran to completion (the initial 8 claimed at launch; tail jobs 25/21/11/30 claimed 01:42–01:45). Jobs whose children were forked **into** the starved window aborted.

Strongly supported (not directly measured): the underlying cause is **container thread/process-count exhaustion**, not memory. Supporting evidence: (a) the aborts are immediately preceded and surrounded by `pthread_create` / `posix_spawn` **EAGAIN** errors — the signature of hitting a thread/process limit (`RLIMIT_NPROC` , cgroup `pids.max` , or `kernel.threads-max`); (b) parent RSS stayed flat at ~200 MB, ruling out parent memory pressure; (c) under 8 concurrent isolated children, each launching Chromium (which spawns dozens of threads) for source fetching, aggregate thread demand is highest exactly in the mid-run window where the aborts cluster.

Not directly measured: the specific limit that was hit (`RLIMIT_NPROC` vs cgroup `pids.max` vs `threads-max`) and the live thread/pid count inside the worker container at the moment of abort are not instrumented and were not captured — the `rss_mb` diagnostic is **parent-process only**; the container and the isolated children are not memory- or thread-instrumented. This is the one gap between “strongly supported” and “measured”. No fix is proposed (no code/settings changes are authorised); the evidence is reported as-is.

This failure mode did **not** occur in the 10-company acceptance run (batch 1275), which never reached this concurrency-driven thread demand. It is a load-dependent failure that appears at the 25-company / 8-slot scale.

---

8. Browser-fetch failures and extraction outcomes

Source acquisition degraded during the same saturation window, and the pipeline’s fallback machinery absorbed most of it.

Signal	Count (full run, merged logs)
Chromium launch failures / circuit-breaker-open events	231 (`Chromium launch attempt failed` , `Chromium launch circuit OPEN`)
`pthread_create` / `posix_spawn` / `Cannot fork` (resource-driven)	139
Per-URL fetch failures (HTTP / direct-PDF / force-headless / transient)	~70 (csx.com, darden, publicnow, celestica, idemitsu, southerncompany, magna, stoxx, bce, barrattredrow, …)
Fetch timeouts	116
Fallback / recovery events	**283** (`fallback` 174, `recovered N chars` 71, `firecrawl` 38)
Explicit “unrecoverable source / gave up” markers	**0**

The Chromium fetcher tripped its **circuit breaker** during the thread-starved window and the pipeline fell back to non-browser fetch strategies (Firecrawl / direct HTTP), which recovered content 283 times. Each of the 12 completed companies had its research corpus frozen (`CORPUS SNAPSHOT: frozen for batch 1276`) with per-company breadth as follows:

Company (completed)	Corpus docs frozen
MAGNA INTERNATIONAL	53
CELESTICA	41
DARDEN RESTAURANTS	34
Power Assets Holdings	34
BCE	29
CSX	27
Idemitsu Kosan	26
BARRATT REDROW	26
Dominion Energy	24
AXA	22
THE SOUTHERN COMPANY	17
OTSUKA CORPORATION	11

Honest caveat on source completeness (per instruction — 36 score rows ≠ comprehensive research): no explicit “source unrecoverable” log line was emitted, but the absence of such a marker does **not** prove zero source gaps. Source acquisition for the completed companies ran partly while the Chromium circuit breaker was open, so some browser-only sources were substituted by HTTP/Firecrawl fallbacks or omitted silently. The thinner corpora — notably **OTSUKA (11 docs)** and **THE SOUTHERN COMPANY (17 docs)** — are consistent with degraded source acquisition during the saturation window and should **not** be read as confirmed-comprehensive. The 432 score rows are complete per-measure, but per-source research breadth is uneven and, for the thinner corpora, below what an uncongested run would likely have gathered.

---

9. Cost and wall-clock vs boundaries

Boundary	Observed	Limit	Status
Recorded-model monitoring cost	**$5.1375** (17,847,084 tokens, deepseek-chat)	$14	Within
Wall-clock	**≈ 53.3 min**	90 min	Within

Cost accrued only from the 12 completed companies plus the (near-instant, pre-abort) failed spawns; the SIGABRT failures consumed negligible model spend because the children died before doing LLM work.

---

10. Stop-condition assessment

Stop condition	Observed?	Note
≥2 retry-exhausted job failures (S4)	**YES — tripped**	13 retry-exhausted failures; this is the governing stop condition for the run
Lock-renewal failure	No	renewals firing ~75 s throughout
Stalled redelivery	No	—
Overlapping stale execution	No	—
Unreliable cleanup	No	permits returned to 0; no orphaned children
Permit leak (held not returning to 0)	No	returned to `held=0 waiting=0`
Parent crash / contamination	No	flat RSS, continuous heartbeat
Cost ≥ $14	No	$5.14
Wall ≥ 90 min	No	~53 min

The run was carried to its natural terminal state (`pending_review`) rather than aborted early; the dominant systemic condition is the **isolated-child spawn abort under load**, a new failure mode distinct from the recovered provider/browser errors catalogued in §8.

---

11. Settlement verification (post-terminal)

- Batch 1276 stable at `pending_review`, `completed_jobs=12`, `failed_jobs=13`; **no non-terminal jobs remain** for this batch.
- LLM permits settled at `held=0 waiting=0 max=8` (sampled to 02:22:25Z).
- No orphaned isolated children; no late `[WORKER-ISOLATION]` lines; no late SIGABRT after 01:47:14.
- No immutable snapshot written (by design — a batch with terminal failures enters `pending_review` and does not save results).
- Parent supervisor healthy at settlement.

---

12. Conclusion

- **Clean completion: FAILED** (12/25 first-attempt; 13 retry-exhausted SIGABRT-spawn failures; no 25-company snapshot).

- **Isolation containment: RELIABLE** — the boundary held; the supervisor, permits, and the rest of the batch were never contaminated; failures were caught and driven to a correct terminal state.
- **Isolated-child execution path: UNRELIABLE under the 25-company / 8-slot load** — 13 children self-aborted at bootstrap (SIGABRT, `stage=spawn`), mechanism confirmed, underlying cause strongly supported as container thread/process-count exhaustion (not memory), with the exact limit not directly instrumented.
- **Cost $5.14 and wall ~53 min** stayed well within the $14 and 90-minute boundaries.
- **No rerun was performed. No code, replica, or settings change was made. The run is paused pending your direction**, per the approval terms.